

Année scolaire 2026-2027

Lycée Victor Hugo - Colomiers

Première NSI

AP

Algorithmes et Programmation

Thomas Ricart

`ricart.lvh@laposte.net`

`https://thomasricart.github.io`

6 juin 2026

AP01

CONSTRUCTIONS ÉLÉMENTAIRES

I. Les variables en Python

Déclarer une variable en Python

Pour déclarer une variable en Python, il suffit de lui donner un nom et de lui affecter une valeur à l'aide du symbole `=`. Par exemple, pour déclarer une variable `x` qui contient la valeur 10, on écrit :

</> Code Python

```
x = 10
```

Affecter à `x` la valeur 10

Le nom des variables en Python

- Elles doivent commencer par une lettre ou un underscore (`_`).
- Elles peuvent contenir des lettres, des chiffres et des underscores.
- Elles ne peuvent pas être des mots réservés du langage Python (comme `if`, `for`, `while`, etc.).
- `Variable` et `variable` sont considérées comme deux variables différentes.

Exemple : Noms de variables valides et invalides

Bonne variable	<code>ma_variable</code>
Mauvaise variable	<code>Ma_variable</code> , <code>ma variable</code> , <code>1_variable</code> , <code>variable@</code>
A éviter	Variable non explicites (<code>a</code> , <code>x</code> , <code>u</code> etc)

Afficher une variable en Python

Pour afficher la valeur d'une variable en Python, on utilise la fonction `print()`. Par exemple, pour afficher la valeur de la variable `x`, on écrit :

</> Code Python

```
print(x)
```

Afficher la valeur de `x`

Les types de variables en Python

- **int** : pour les nombres entiers (ex : 1, 42, -5)
- **float** : pour les nombres à virgule flottante (ex : 3.14, -0.01). La virgule est représentée par un point.
- **str** : pour les chaînes de caractères (ex : "Bonjour", 'Python')
- **bool** : pour les valeurs booléennes (ex : True, False)

```
# type (int)
a = 5
b = 6
```

```
# type (str)
a = 'Bonjour'
b = '6'
```

```
# type (float)
a = 5.0
b = 6.8
```

```
# type (bool)
a = True
b = False
```

Opérations mathématiques de base en Python

+ addition : 5 + 3 donne 8

- soustraction : 5 - 3 donne 2

* multiplication : 5 * 3 donne 15

/ division : 6 / 2 donne 3.0 (float)

** puissance : 2 ** 3 donne 8

% reste de la division entière : 7 % 3 donne 1

// quotient de la division entière 7 // 3 donne 2

```
>>> print(17+2)
19
>>> print(17-2)
15
```

```
>>> print(17*2)
34
>>> print(17/2)
8.5
```

```
>>> print(17%2)
1
>>> print(17//2)
8
```

Exercice 1 : Repérer les les types de variables et leurs valeurs dans les programmes suivants

- `x = 5`
- `y = "Hello"`
- `z = True`
- `a = 3.14`
- `c = 6/2`
- `d = 6//2`
- `f = '1' + '2'`
- `g = 2**3`

II. Les variables de type chaîne de caractère `str`

Les chaînes de caractères

Les chaînes de caractères sont des variables qui contiennent du texte. Elles sont délimitées par des guillemets simples ou doubles.

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a + b
>>> print(c)
Bonjour a tous
```

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a * 3
>>> print(c)
BonjourBonjourBonjour
```

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a[0]
>>> print(c)
B
```

Dans l'exemple ci-dessus, la variable `c` contient la concaténation de `a` et `b` dans le premier cas, la répétition de `a` trois fois dans le deuxième cas et le premier caractère de `a` dans le troisième cas.

indexation des chaînes de caractères

Les chaînes de caractères peuvent être vues comme un ensemble de caractères « collés » les uns aux autres. Chaque caractère d'une chaîne de caractères est indexé : il y a un index qui donne sa position dans la chaîne.

Indice	0	1	2	3	4	5	6
Caractère	B	o	n	j	o	u	r
Indice Négatif	-7	-6	-5	-4	-3	-2	-1

```
>>> mot = 'Bonjour'
>>> mot[0]
'B'
>>> mot[4]
'o'
```

```
>>> mot = 'Bonjour'
>>> n = len(mot)
>>> n
7
>>> mot[-1]
'r'
```

Les opérations sur les chaînes de caractères

- + pour la concaténation : `'Bonjour' + ' a tous'` donne `'Bonjour a tous'`
- * pour la répétition : `'Bonjour' * 3` donne `'BonjourBonjourBonjour'`
- [] pour l'indexation : `'Bonjour'[0]` donne `'B'`
- in pour le test d'appartenance : `'on' in 'Bonjour'` donne `True`
- not in pour le test d'appartenance : `'on' not in 'Bonjour'` donne `False`
- len() pour la longueur : `len('Bonjour')` donne `7`

```
>>> mot1 = 'Bonjour'
>>> mot2 = 'Thomas'
>>> mot1 + ' ' + mot2
'Bonjour Thomas'
```

```
>>> 'j' in 'Bonjour'
True
>>> 'b' in 'Bonjour'
False
```

```
>>> 'abracadabra'[4]
'a'
>>> len('abracadabra')
11
```

Opérations avancées sur les chaînes de caractères : le Slicing

[a:b]	conserve les caractères d'indice [a:b[: 'Bonjour' [1:4] donne 'onj'
[a:]	conserve les caractères d'indice [a:fin] : 'Bonjour' [3:] donne 'jour'
[:b]	conserve les caractères d'indice [debut:b[: 'Bonjour' [:3] donne 'Bon'
[::-1]	inverse les caractères : 'Bonjour' [::-1] donne 'ruojnoB'
[1:-1]	élimine le premier et dernier caractère : 'Bonjour' [1:-1] donne 'onjou'

🔗 Exercice 2 : Prédire ce que vont afficher les codes suivants

```
>>> mot = 'Numerique'
>>> n = len(mot)
>>> mot[5]
>>> mot[n]
>>> mot[-1]
>>> mot[8]
```

```
>>> mot = 'Numerique'
>>> m in mot
>>> e not in mot
>>> mot+mot[2]
>>> mot*2
>>> 'abc'+mot
```

```
>>> mot = 'Numerique'
>>> mot[1:5]
>>> mot[:4]
>>> mot[3:]
>>> mot[1:-1]
```

Principe du transtypage

Les types des variables en python sont dynamiques, on peut modifier sa valeur ainsi que son type.

```
>>> a = '5'      # a est de type str
>>> b = int(a)   # b est de type int
>>> print(b)
5
```

```
>>> a = 4        # a est de type int
>>> b = str(a)   # b est de type str
>>> print(b)
4
```

III. Les variables de type booléen bool

Les booléens

Les booléens sont des variables qui ne peuvent prendre que deux valeurs : True (vrai) ou False (faux).

Les booléens peuvent être utilisés dans des opérations logiques.

```
>>> a = True
>>> c = not a
>>> c
False
```

```
>>> a = True
>>> b = False
>>> c = a and b
>>> c
False
```

```
>>> a = True
>>> b = False
>>> c = a or b
>>> c
True
```

Table de vérité du not

Expression	Résultat
not True	False
not False	True

Table de vérité du and

Expression	Résultat
True and True	True
True and False	False
False and True	False
False and False	False

Table de vérité du or

Expression	Résultat
True or True	True
True or False	True
False or True	True
False or False	False

Booléen comme résultat de comparaison

Les booléens sont le résultat d'une comparaison. Les opérateurs de comparaison sont :

== pour le test d'égalité	> pour le test de supériorité
!= pour le test de différence	<= pour le test d'infériorité ou d'égalité
< pour le test d'infériorité	>= pour le test de supériorité ou d'égalité

Booléen comme résultat de comparaison

```
>>> a = 5
>>> b = 6
>>> c = (a == b)
>>> c
False
```

```
>>> a = 5
>>> b = 6
>>> c = (a != b)
>>> c
True
```

```
>>> a = 5
>>> b = 6
>>> c = (a < b)
>>> c
True
```

Exercice 3

Quelle est la valeur de la variable c après l'exécution des codes suivants ?

```
>>> a = 3
>>> b = 1
>>> c = a > b
```

```
>>> a, b = True, False
>>> c = (a and b) or (not a)
```

```
>>> a = True
>>> b = False
>>> c = (a or b) and (not a)
```

```
>>> a = True
>>> b = False
>>> x, y = 7, 9
>>> c = (b or (x == y)) or (a and (x != y))
```

AP02

LES FONCTIONS

I. La gestion des entrées - sorties**Gestion des entrées en Python**

Il est possible en python de demander à l'utilisateur d'entrer une valeur au clavier. Cela s'effectue via la commande `input`. La valeur tapée par l'utilisateur est convertie au format chaîne de caractère (str)

</> Code Python

```
a = input("Entrer une valeur au clavier")      # a sera de type str
b = int(input("Entrer une valeur au clavier"))  # b sera de type int
```

Méthode 1 : Affichage dans la zone programme

</> Code Python

```
1 def f(x):
2     y = x ** 2  # Fonction calculant x**2
3     return y
4
5 f(4)           # PAS D'AFFICHAGE: f(4) est créé mais est perdue
6 print(f(4))    # AFFICHAGE: f(4) créé puis affiché. La valeur est perdue
7 carre = f(6)   # PAS D'AFFICHAGE: f(6) créé puis sauvegardé dans carre.
8 print(carre)   # AFFICHAGE: affichage de la valeur de la variable carre
```

On peut afficher des chaînes complexes avec le séparateur virgule ,

</> Code Python

```
a, b = 6, 4
print("La somme de", a, "et", b, "est", a + b)
```

Méthode 2 : Affichage dans la console python

La console python accepte en plus de la commande `print` un affichage direct en appelant la variable.

```
>>> a = 6
>>> a
6
```

Méthode 3 : Formatage de l'affichage avec f-string et `format()` (optionnel)

L'idée est de créer un texte à trous que l'on va compléter par des variables.

```
>>> name = 'Paul'
>>> age = 25
>>> print(f'Votre nom est {name} et vous avez {age} ans.')
Votre nom est Paul et vous avez 25 ans.
```

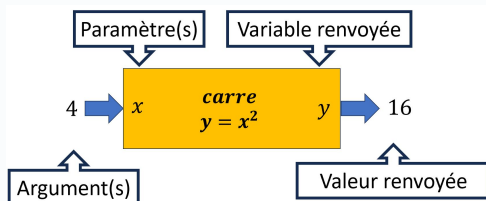
```
>>> print('Votre nom est {} et vous avez {} ans'.format(name, age))
Votre nom est Paul et vous avez 25 ans
```

II. Les fonctions

Présentation

Une **fonction** en python est un objet qui se comporterait comme une fonction mathématique. On peut prendre pour exemple une fonction f définie par $f(x) = x^2$.

- f est le nom de la fonction
- x est un paramètre (une variable en mathématique). C'est juste un concept dont on va se servir pour appliquer la fonction. Ce nom de variable n'est connu qu'à l'intérieur de la fonction.
- x^2 est l'expression de la fonction. C'est la manière dont on va calculer l'image de x par la fonction f .



- Ici la valeur 4 est un argument, c'est à dire la valeur sur laquelle on va appliquer la fonction.
- La valeur 16 est la valeur renvoyée par la fonction. Ne pas confondre variable renvoyée et valeur renvoyée

</> Code Python

```

1 def f(x):
2     """ Fonction renvoyant x ** 2 """
3     y = x ** 2      # Calcul de x ** 2
4     return y        # valeur renvoyée: y

```

- **Ligne 1** : def permet de définir le nom de la fonction. Ici la fonction se nomme f.
- **Ligne 1** : f(x) permet de définir le nom et le nombre des paramètres. Ici la fonction f a besoin d'un seul paramètre x
- **Ligne 2** : commentaire permettant de décrire ce que fait la fonction. Cette ligne n'est pas interprétée
- **Ligne 3** : on lit de la droite vers la gauche : « on prend le contenu de la variable x, on le met au carré et on l'affecte à une nouvelle variable appelée y »
- **Ligne 4** : on renvoie la valeur de la variable y.

On vient de définir une nouvelle fonction. Il faut maintenant lui donner un argument pour qu'elle puisse calculer une valeur et la renvoyer. C'est ce que l'on appelle **appeler** la fonction.

</> Code Python

```

a = 4      # on affecte 4 à la variable a
z = f(4)    # on appelle la fonction f avec comme argument 4
           # et on récupère la valeur renvoyée dans la variable z

```

- la commande return renvoie la valeur contenue dans y
- elle provoque la sortie immédiate de la fonction
- si possible on ne met qu'un seul return dans la fonction
- il peut n'y avoir aucun return, on parle alors de procédure

Prototypage d'une fonction

</> Code Python

```

1 def ma_fonction(param1, param2, ...):
2     ''' Docstring: Explication de ce que fait ma_fonction
3     : description des paramètres et leurs types
4     : description des valeurs de sortie et leurs types '''
5     # Corps de la fonction
6
7     return valeur    # La variable 'valeur' est renvoyée

```

Exemple : Exemple de fonction Python

Voici un exemple d'une fonction qui fait quelque chose de très simple : elle prend deux valeurs en paramètres (a et b) et renvoie la somme des ces deux valeurs. Cette somme a été créée et affectée à la variable somme.

</> Code Python

```
1 def calc_somme(a, b):
2     ''' Fonction calculant la somme de deux valeurs
3     : a et b deux entiers (int)
4     : valeur renvoyée est un entier (int) '''
5     somme = a + b      # le résultats de a + b est affecté à la variable somme
6     return somme      # la valeur de somme est renvoyée par la fonction
```

Il est maintenant possible de se servir de cette fonction.

</> Code Python

```
s = calc_somme(6, 8)    # s récupère la valeur renvoyée par calc_somme(6, 8)
print(s)               # la valeur de la variable s est affichée (14)
```

Remarque : Variables locales et variables globales

- les **variables locales** ne sont connues que dans la fonction
- les **variables globales** sont connues dans tout le programme

</> Code Python

```
print(a)    # ERREUR -> a est une variable locale (tout comme b et somme)
```

Il est donc tout à fait normal que les variables a, b et somme ne soient pas connues à l'extérieur de la fonction. Il est de bon usage de ne pas appeler de la même façon des variables locales et des variables globales.

Vérification de la valeur d'une variable

La fonctionnalité assert vérifie qu'une affirmation est vraie. Si c'est le cas, rien ne se passe, sinon un message d'erreur est donné

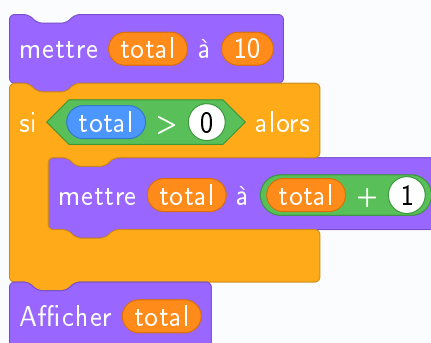
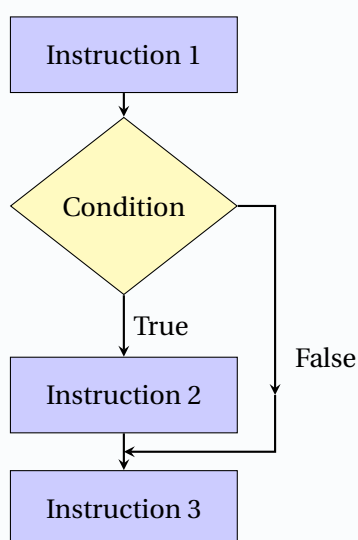
</> Code Python

```
assert calcule_somme(6, 8) == 14      # True -> aucune message
assert calcule_somme(6, 8) == 12      # False -> message d'erreur
assert not(calcule_somme(6, 8) == 12) # True -> aucun message
```


AP03

CONDITIONS IF

Les structures conditionnelles IF



```
# En python
Instruction_1
if condition:
    Instruction_2
Instruction_3
```

```
# En python
total = 10
if total > 0:
    total = total + 1
print(total)
```

- l'Instruction 1 est effectuée
- SI la Condition est True alors l'Instruction 2 est effectuée
- SINON on va directement à l'Instruction 3
- Dans tous les cas l'Instruction 3 est effectuée

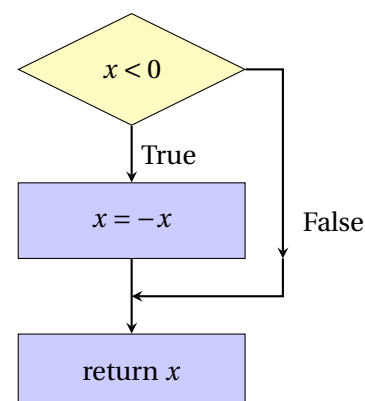
La Condition est un test qui est évalué en python par un booléen.

Exemple : La condition IF sans ELSE : la valeur absolue d'un nombre

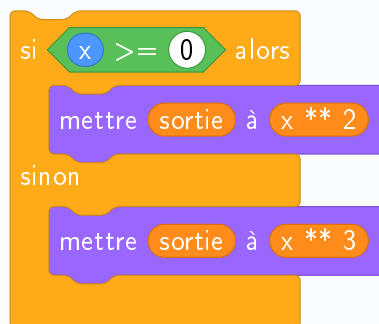
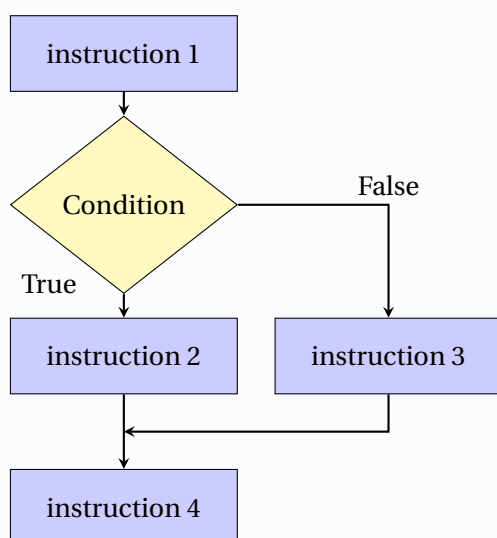
- si $x \geq 0$, alors $|x| = x$
- si $x < 0$, alors $|x| = -x$

</> Code Python

```
1 def valeur_absolue(x):
2     ''' Renvoie |x| '''
3     if x < 0:
4         # si x < 0 on échange x par -x
5         x = - x
6     return x
```



Les structures conditionnelles IF – ELSE



```
# En python
instruction_1
if condition:
    instruction_2
else:
    instruction_3
instruction_4
```

- l'instruction 1 est effectuée
- SI la Condition est True alors l'instruction 2 est effectuée
- SINON l'instruction 3 est effectuée
- Dans tous les cas l'instruction 4 est effectuée

Exemple : La condition IF – ELSE

</> Code Python

```

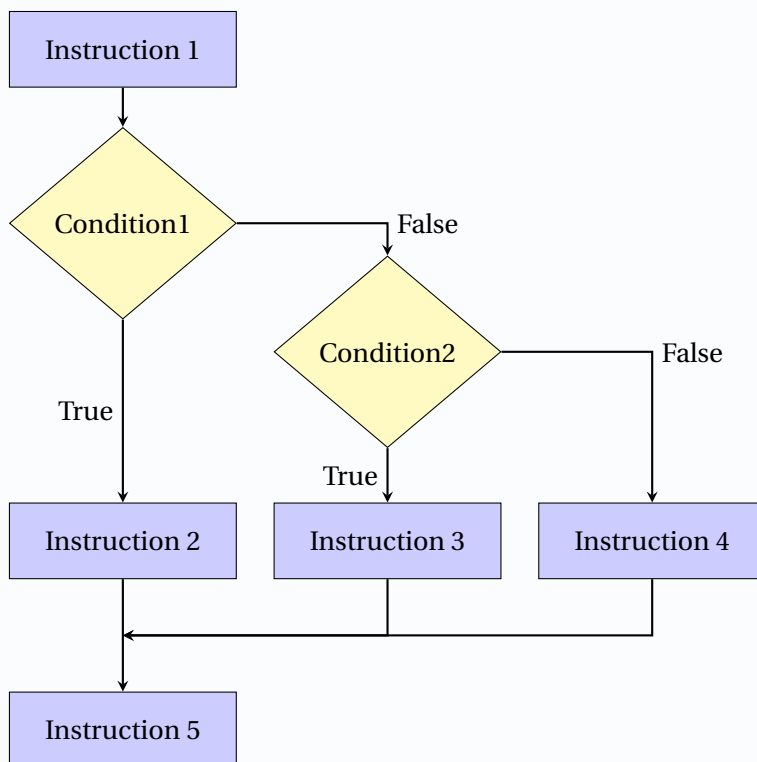
1 def ma_fonction(x):
2     if x >= 0:
3         sortie = x ** 2
4     else:
5         sortie = x ** 3
6     return sortie

```

**Les structures conditionnelles IF – ELIF – ELSE**

Dans le cas où il y a plusieurs possibilités de réponse, on peut utiliser deux structures différentes en python. On suppose une fonction prenant comme paramètre une variable x de type entier.

$$f(x) = 1 \text{ si } x > 0 \quad f(x) = 0 \text{ si } x = 0 \quad f(x) = -1 \text{ si } x < 0$$



</> Code Python

```

1 def ma_fonction(x):
2     if x > 0:
3         sortie = 1
4     elif x == 0:
5         sortie = 0
6     else:
7         sortie = -1
8     return sortie

```

</> Code Python

```

1 def ma_fonction(x):
2     if x > 0:
3         sortie = 1
4     else:
5         if x == 0:
6             sortie = 0
7         else:
8             sortie = -1
9     return sortie

```

Exercice 1 : Compléter les fonctions suivantes

</> Code Python

```

1 def plus_grand2(a, b):
2     ''' Renvoie le plus grand des
3         deux nombres a et b '''
4
5
6
7
8
9
10
11
12 assert plus_grand2(3, 5) == 5
13 assert plus_grand2(3, -5) == 3

```



</> Code Python

```

1 def plus_grand3(a, b, c):
2     ''' Renvoie le plus grand des
3         trois nombres a, b et c '''
4
5
6
7
8
9
10
11
12 assert plus_grand3(3, 5, 2) == 5
13 assert plus_grand3(3, -5, 2) == 3

```



AP04

BOUCLES FOR ET WHILE

I. La boucle FOR

Les boucles FOR

La boucle FOR est une boucle bornée. On sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
Pour i allant de 0 a 6:
    instruction2
instruction3
```

- instruction1 est effectuée
- i est un itérateur qui va aller de 0 à 6, c'est à dire 7 tours
- On effectuera donc 7 fois l'instruction2
- Quand on a fini la boucle on effectue l'instruction3

La boucle FOR en python

</> Code Python

```
for i in range(0, 7):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4 5 6
```

</> Code Python

```
for i in range(2, 7):
    print(i, end=" ")
# affichage:
# 2 3 4 5 6
```

</> Code Python

```
for i in range(5):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4
```

Prototypage de la boucle FOR

La boucle FOR s'écrit de manière générale de la façon suivante :

</> Code Python

```
for i in range(a, b):
```

La boucle est effectuée $b - a$ fois et la variable i prend pour valeur tous les entiers entre $[a; b[$

Exemple : Trace de la structure FOR

</> Code Python

```
for i in range(2, 6):
    print(i)
print('fin')
```

i	i < 6	instruction
2	True	print(2)
3	True	print(3)
4	True	print(4)
5	True	print(5)
6	False	print('fin')

Itération sur une chaîne de caractères via les indices

Une chaîne de caractères str est un itérateur, c'est à dire qu'il est possible d'itérer sur ses éléments.

```
>>> mot = 'Hello'
>>> mot[1]
'e'
```

</> Code Python

```
mot = 'Hello'
n = len(mot)
for i in range(0, n):
    print(mot[i])
print('Fin')
```

Indice	0	1	2	3	4
Caractère	H	e	l	l	o
Indice Négatif	-5	-4	-3	-2	-1

i	i < n	instruction	affichage
0	True	print(mot[0])	H
1	True	print(mot[1])	e
2	True	print(mot[2])	l
3	True	print(mot[3])	l
4	True	print(mot[4])	o
5	False	print('Fin')	Fin

Itération sur une chaîne de caractères via les éléments

On peut itérer directement sur les éléments qui constituent cette chaîne de caractère
On crée un variable `lettre` par exemple qui prendra pour valeur chacun des caractères de mot.

</> Code Python

```
mot = 'Hello'
for lettre in mot:
    print(lettre)
print('Fin')
```

lettre	instruction	affichage
'H'	<code>print(lettre)</code>	H
'e'	<code>print(lettre)</code>	e
'l'	<code>print(lettre)</code>	l
'l'	<code>print(lettre)</code>	l
'o'	<code>print(lettre)</code>	o
None	<code>print('Fin')</code>	Fin

- Il n'y a pas besoin de connaître la longueur du mot
- Il n'y plus l'utilisation d'indice
- `lettre` prend pour valeur chaque caractère de mot
- lorsque `lettre` a pris pour valeur tous les caractères de mot, la boucle s'arrête.

🔪 Exercice 1

Écrire une fonction `entier(n)` permettant d'afficher tous les entiers entre 0 et n inclus

🔪 Exercice 2

Écrire une fonction `entier(a,b)` qui affiche tous les entiers entre a et b inclus

🔪 Exercice 3

Écrire une fonction qui affiche chaque lettre d'un mot en itérant sur les indices

🔪 Exercice 4

Écrire une fonction qui affiche chaque lettre d'un mot en itérant sur les caractères

Algorithmes de Recherche dans un itérable

Cet algorithme permet de savoir si un élément est présent dans un itérable.

- Le mot sera la variable `mot`
- La lettre que l'on cherche sera la variable `lettre_cherchee`

On peut utiliser l'algorithme suivant :

1. on parcourt `mot` à l'aide d'une boucle FOR
2. on observe donc chaque caractère `mot[i]`
3. si le caractère est égal à `lettre_cherchee`, alors on renvoie la valeur `True` et on arrête la boucle.
4. sinon on ne fait rien et on revient à l'étape 2 pour observer le caractère suivant.
5. si on a observé tous les caractères de mot sans trouver `lettre_cherchee`, on renvoie `False`

🔪 Exercice 5 : Écrire une fonction `est_present(mot, lettre_cherchee)`

</> Code Python

```
def est_present(mot, lettre_cherchee):
    ''' Renvoie True si lettre_cherchee est dans mot et False sinon'''

    assert est_present('abracadabra', 'a') == True
    assert est_present('abracadabra', 'z') == False
```

Exercice 6

Ecrire la fonction `est_present2(mot, lettre_cherchee)` qui renvoie l'indice de la première occurrence de `lettre_cherchee` dans `mot` ou `-1` si elle n'est pas présente

Algorithmes de comptage dans un itérable

Cet algorithme permet de compter le nombre de fois qu'un élément est présent dans un itérable.

- Le mot sera la variable `mot`
- La lettre que l'on cherche sera la variable `lettre_cherchee`

On peut utiliser l'algorithme suivant :

1. on initialise un compteur à 0
2. on parcourt `mot` à l'aide d'une boucle FOR
3. on observe donc chaque caractère `mot[i]`
4. si le caractère est égal à `lettre_cherchee`, alors on incrémente le compteur.
5. sinon on ne fait rien
6. on revient à l'étape 3 pour observer le caractère suivant.
7. si on a observé tous les caractères de `mot`, on renvoie le compteur

Exercice 7 : Écrire une fonction `compte(mot, lettre_cherchee)`**</> Code Python**

```
def compte(mot, lettre_cherchee):  
    ''' Renvoie le nombre de fois que lettre_cherchee est dans mot '''  
  
  
  
assert compte('abracadabra', 'a') == 5  
assert compte('abracadabra', 'z') == 0
```

Exercice 8 : Ecrire la fonction `compte2(mot, lettre_cherchee)` qui itère sur les éléments**</> Code Python**

```
def compte2(mot, lettre_cherchee):  
    ''' Renvoie le nombre de fois que lettre_cherchee est dans mot '''  
  
  
  
assert compte2('abracadabra', 'a') == 5  
assert compte2('abracadabra', 'z') == 0
```

II. La boucle WHILE

Les boucles WHILE

La boucle WHILE est une boucle non bornée. On ne sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
tant que condition1 vraie:
    instruction2
instruction3
```

- instruction1 est effectuée
- tant que la condition1 est vraie on effectue l'instruction 2
- quand la condition1 est fausse, on sort de la boucle;
- on effectue l'instruction3

Prototypage de la boucle WHILE en python

condition1 est un booléen qui est évalué à chaque tour de boucle. Tant que condition1 True, on effectue les instructions de la boucle. Lorsque condition1 devient False, on sort de la boucle.

</> Code Python

```
while condition1:
    instruction
```

III. Exercices

🔪 Exercice 9 : Dire ce qu'affichent les programmes suivants et expliquer pourquoi

</> Code Python

```
1 n = 0
2 u = 1
3 while u < 1000:
4     n = n + 1
5     u = u * 2
6 print(n)
7 print(u)
```

</> Code Python

```
1 mot = 'abracadabra'
2 c = 0
3 trouve = False
4 while c < len(mot) and not(trouve):
5     trouve = mot[c] == 'r'
6     c = c + 1
7 print(trouve)
```

🔪 Exercice 10 : Boucle FOR

1. Écrire une fonction somme(n) qui renvoie la somme des entiers de 1 à n inclus. Par exemple, somme(5) doit renvoyer 1 + 2 + 3 + 4 + 5 = 15.

</> Code Python

```
def somme(n):
    ''' Renvoie la somme des entiers de 1 à n inclus'''

    assert somme(5) == 15
    assert somme(10) == 55
```

2. Écrire une fonction `factorielle(n)` qui renvoie la factorielle de n . Par exemple, `factorielle(5)` doit renvoyer $1 \times 2 \times 3 \times 4 \times 5 = 120$.

</> Code Python

```
def factorielle(n):  
    ''' Renvoie la factorielle de n '''  
  
  
  
assert factorielle(5) == 120  
assert factorielle(10) == 3628800
```

3. Ecrire une fonction `replace_lettre(mot, lettre_a_remplacer, nouvelle_lettre)` qui remplace toutes les occurrences de `lettre_a_remplacer` par `nouvelle_lettre` dans `mot`. Par exemple, `replace_lettre('abracadabra', 'a', 'o')` doit renvoyer `'obrocodobro'`.

</> Code Python

```
def replace_lettre(mot, lettre_a_remplacer, nouvelle_lettre):  
    ''' Renvoie mot dans lequel toutes les occurrences de  
    lettre_a_remplacer ont été remplacées par nouvelle_lettre '''  
  
  
  
assert replace_lettre('abracadabra', 'a', 'o') == 'obrocodobro'  
assert replace_lettre('abracadabra', 'b', 'x') == 'axracadaxra'
```

4. Ecrire une fonction `supprime_lettre(mot, lettre_a_supprimer)` qui supprime toutes les occurrences de `lettre_a_supprimer` dans `mot`. Par exemple, `supprime_lettre('abracadabra', 'a')` doit renvoyer `'brcdbr'`.

</> Code Python

```
def supprime_lettre(mot, lettre_a_supprimer):  
    ''' Renvoie mot dans lequel toutes les occurrences de  
    lettre_a_supprimer ont été supprimées '''  
  
  
  
assert supprime_lettre('abracadabra', 'a') == 'brcdbr'  
assert supprime_lettre('abracadabra', 'b') == 'aracadara'
```

Exercice 11 : Boucle While

- On dispose d'une feuille de papier d'épaisseur 0,1 mm. Combien de fois doit-on la plier au minimum pour que l'épaisseur dépasse la hauteur de la tour Eiffel 324 m. Écrire une fonction `nb_plis()` en Python pour résoudre ce problème.
- Inès veut construire une pyramide à base carrée comme sur la photo. La pyramide sur la photo a 7 étages. Inès a n billes. Combien d'étages au maximum aura sa pyramide? Écrire une fonction `nb_etages(n)` en Python pour répondre au problème.



</> Code Python

```
1 assert nb_etages(1000) == 14
```

- L'ordinateur tire un nombre entier au hasard entre 0 et 100. L'utilisateur doit le trouver et pour cela propose des valeurs. L'ordinateur indique pour chaque valeur proposée si la valeur est trop petite, trop grande ou s'il a trouvé. Écrire une fonction `jeu()` en Python pour jouer à ce jeu. En combien de coups est-on sûr de trouver?

Méthode 1 : La bibliothèque random

</> Code Python

```
import random          # choisit un nombre entre 0 et 100
nombre_aleatoire = random.randint(0, 100)
```

- En mathématiques, la suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence généralement par les termes 0 et 1 et ses premiers termes sont 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.
 - Écrire une fonction `fibonacci(n)` qui renvoie le terme de rang n de la suite de Fibonacci. (le premier terme est de rang 0)

</> Code Python

```
assert fibonacci(0) == 0
assert fibonacci(1) == 1
assert fibonacci(2) == 1
assert fibonacci(9) == 34
```



- Écrire une fonction `rang_fibonacci()` qui renvoie le rang du premier terme supérieur ou égal à 10000.

</> Code Python

```
assert rang_fibonacci() == 21
```

- L'utilisateur donne un entier positif et le programme annonce combien de fois de suite cet entier est divisible par 2. Écrire la fonction `divise_par_2(n)` qui renvoie ce nombre de fois. Par exemple, `divise_par_2(48)` doit renvoyer 4 car $48 = 2^4 \times 3$ et 4 est le plus grand entier tel que 48 est divisible par 2^4 .

</> Code Python

```
assert divise_par_2(48) == 4
assert divise_par_2(7) == 0
```



- Écrire une fonction `tirage()` qui fait des tirages successifs d'entiers 0 ou 1 jusqu'à tirer 3 fois 1 consécutivement et renvoie le nombre de tirages qui ont été nécessaires.

AP05

LISTES ET TUPLES

I. Les listes

Définition 1 : Les listes

Une **liste** est une structure de donnée qui contient des éléments ou items identifiés par des indices.

Ces éléments sont :

- **ordonnés** : les éléments d'une liste sont rangés dans un ordre précis. Chaque élément a une position dans la liste, appelée indice.
- **séparés par des virgules** : les éléments d'une liste sont séparés par des virgules.
- **entre crochets** : les éléments d'une liste sont entourés de crochets [] en python.

Indice i	0	1	2	3	4	5	6	7
Élément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Créer une liste

</> Code Python

```
1 ma_liste = [1, 2, 3, 4, 5]
```

</> Code Python

```
1 ma_liste = ['a', 'b', 'c']
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(type(ma_liste))           # Une liste est de type 'list'
<class 'list'>
```

Accéder aux éléments d'une liste

Pour accéder à un élément d'une liste, on utilise son indice. Les indices commencent à 0.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(ma_liste[0])
1
>>> print(ma_liste[2])
3
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(ma_liste[-1])
5
>>> print(ma_liste[-2])
4
```

Modifier les éléments d'une liste

Les éléments d'une liste peuvent être modifiés en utilisant leur indice.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste[0] = 10
>>> print(ma_liste)
[10, 2, 3, 4, 5]
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste[4] = 20
>>> print(ma_liste)
[1, 2, 3, 4, 20]
```

Connaitre le nombre d'éléments d'une liste

Pour connaître le nombre d'éléments d'une liste, on utilise la fonction `len()`.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(len(ma_liste))
5
```

```
>>> ma_liste = ['a', 'b', 'c']
>>> print(len(ma_liste))
3
```

Exercice 1 : Manipulation de base

1. Créer la liste suivante : [4, 8, 7, 9, 4]
2. Afficher le nombre d'éléments de cette liste. (*affiche 5*)
3. Afficher le troisième élément de cette liste. (*affiche 7*)
4. Modifier le deuxième élément de cette liste pour qu'il soit égal à 10. (*affiche [4, 10, 7, 9, 4]*)
5. Modifier le dernier élément de cette liste pour qu'il soit égal à 5. (*affiche [4, 10, 7, 9, 5]*)
6. Modifier le premier éléments de la liste en le remplaçant par la somme du premier et du troisième élément de cette liste. (*affiche [11, 10, 7, 9, 5]*)

Itérer sur une liste avec une boucle FOR

Comme pour les chaînes de caractères, On peut itérer sur les éléments d'une liste à l'aide d'une boucle FOR.

</> Code Python

```
1 ma_liste = [4, 8, 7, 9, 4]
2 for i in range(0, len(ma_liste)):
3     print(ma_liste[i])
```

</> Code Python

```
1 ma_liste = [4, 8, 7, 9, 4]
2 for elt in ma_liste:
3     print(elt)
```

Exercice 2 : Quelques algorithmes sur les listes

1. Ecrire une fonction `somme(liste)` qui renvoie la somme des éléments d'une liste de nombres.

</> Code Python

```
def somme(liste):
    ''' Renvoie la somme des éléments de liste'''



    assert somme([1, 2, 3, 4, 5]) == 15
    assert somme([4, 8, 7, 9, 4]) == 32
```

2. Ecrire une fonction `maximum(liste)` qui renvoie le maximum des éléments d'une liste de nombres.

</> Code Python

```
def maximum(liste):
    ''' Renvoie le maximum des éléments de liste'''



    assert maximum([1, 2, 3, 4, 5]) == 5
    assert maximum([4, 8, 7, 9, 4]) == 9
```

3. Ecrire une fonction `est_present(elt, liste)` qui renvoie `True` si l'élément `elt` est présent dans la liste `liste`, et `False` sinon.

</> Code Python

```
def est_present(elt, liste):
    ''' Renvoie True si elt est dans liste et False sinon'''

    assert est_present(3, [1, 2, 3, 4, 5]) == True
    assert est_present(6, [1, 2, 3, 4, 5]) == False
```

Liste de listes

Une liste peut contenir des éléments qui sont eux-mêmes des listes. On parle alors de liste de listes.

```
>>> ma_liste = [[1, 2, 3], [4, 5], [7, 8, 9, 10]]
>>> print(len(ma_liste))
3
>>> print(len(ma_liste[1]))
2
>>> print(ma_liste[0])
[1, 2, 3]
>>> print(ma_liste[1])
[4, 5]
>>> print(ma_liste[2])
[7, 8, 9, 10]
>>> print(ma_liste[0][1])
2
>>> print(ma_liste[1][1])
5
>>> print(ma_liste[2][0])
7
```

🔧 Exercice 3 : Manipulation de listes de listes

1. Créer la liste suivante : `[[1, 2], [3, 4], [5, 6]]`
2. Afficher le nombre d'éléments de cette liste. (*affiche 3*)
3. Afficher le deuxième élément de cette liste. (*affiche [3, 4]*)
4. Afficher le premier élément du troisième élément de cette liste. (*affiche 5*)
5. Modifier le deuxième élément du premier élément de cette liste pour qu'il soit égal à 10
6. Afficher la liste modifiée. (*affiche [[1, 10], [3, 4], [5, 6]]*)
7. Modifier le premier élément du deuxième élément de cette liste pour qu'il soit égal à 20
8. Afficher la liste modifiée. (*affiche [[1, 10], [20, 4], [5, 6]]*)
9. Modifier le deuxième élément du troisième élément de cette liste en le remplaçant par la somme du premier et du deuxième élément de ce troisième élément
10. Afficher la liste modifiée. (*affiche [[1, 10], [20, 4], [5, 11]]*)

Exercice 4 : Itérer sur une liste de listes

1. Ecrire une fonction `somme_liste_de_listes(liste)` qui renvoie la somme des éléments d'une liste de listes de nombres.

Code Python

```
def somme_liste_de_listes(liste):  
    ''' Renvoie la somme des éléments de liste'''
```

```
assert somme_liste_de_listes([[1, 2], [3, 4], [5, 6]]) == 21
```

2. Ecrire une fonction `maximum_sous_liste(liste)` qui renvoie la sous liste dont la somme des éléments est maximale.

Code Python

```
def maximum_sous_liste(liste):  
    ''' Renvoie la sous liste dont la somme des éléments est max'''
```

```
assert maximum_sous_liste([[3, 4, 16], [5, 6, 1, 8]]) == [3, 4, 16]
```

Quelques méthodes sur les listes

- `append(elt)` : ajoute l'élément `elt` à la fin de la liste
- `pop()` : supprime et renvoie le dernier élément de la liste
- `pop(i)` : supprime et renvoie l'élément d'indice `i` de la liste

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste.append(6)
>>> print(ma_liste)
[1, 2, 3, 4, 5, 6]
>>> a = ma_liste.pop()
>>> print(a)
6
>>> print(ma_liste)
[1, 2, 3, 4, 5]
>>> b = ma_liste.pop(2)
>>> print(b)
3
>>> print(ma_liste)
[1, 2, 4, 5]
```

Créer une liste par itération

</> Code Python

```
1 ma_liste = []
2 for i in range(0, 5):
3     ma_liste.append(i)
4 print(ma_liste)      # [0, 1, 2, 3, 4]
```

</> Code Python

```
1 ma_liste = []
2 for i in range(0, 5):
3     ma_liste = [i] + ma_liste
4 print(ma_liste)     # [4, 3, 2, 1, 0]
```

Créer une liste par compréhension

</> Code Python

```
1 ma_liste = [i for i in range(0, 5)]
2 print(ma_liste)      # Affiche: [0, 1, 2, 3, 4]
```

Exercice 5 : Quelques méthodes sur les listes

1. Ecrire une fonction `puissance_2(n)` qui renvoie une liste contenant les puissances de 2 de 0 à n inclus.

</> Code Python

```
def puissance_2(n):
    ''' Renvoie une liste contenant les puissances de 2 de 0 à n '''

    #

    assert puissance_2(3) == [1, 2, 4, 8]
    assert puissance_2(5) == [1, 2, 4, 8, 16, 32]
```

2. Ecrire une fonction `est_pair(liste)` qui renvoie une liste contenant les éléments de liste qui sont pairs.

</> Code Python

```
def est_pair(liste):
    ''' Renvoie une liste contenant les éléments pairs'''

    #

    assert est_pair([1, 2, 3, 4, 5]) == [2, 4]
    assert est_pair([1, 3, 5, 7, 9]) == []
    assert est_pair([2, 4, 6, 21, 8, 12, 13, 10]) == [2, 4, 6, 8, 12, 10]
```

3. Ecrire une fonction `supprime_element(liste, element)` qui supprime les éléments d'une liste en partant du dernier jusqu'à l'obtention du premier 0.

</> Code Python

```
def supprimer_element(liste):  
    ''' Supprime les éléments de liste en partant du dernier  
        jusqu'à l'obtention du premier 0 '''  
  
    #  
    assert supprimer_element([0, 0, 1, 1, 0, 1, 1, 1]) == [0, 0, 1, 1, 0]  
    assert supprimer_element([1, 1, 0, 1, 1, 1, 1, 1, 1]) == [1, 1, 0]
```

4. Ecrire une fonction `est_triee(liste)` qui renvoie True si la liste `liste` est triée dans l'ordre croissant, et False sinon.

</> Code Python

```
def est_triee(liste):  
    ''' Renvoie True si liste est triée dans l'ordre croissant et False sinon '''  
  
    #  
  
    assert est_triee([1, 2, 3, 4, 5]) == True  
    assert est_triee([5, 4, 3, 2, 1]) == False
```

Le hasard et les listes

- La fonction `random.choice(liste)` permet de choisir un élément au hasard dans une liste.
- La fonction `random.shuffle(liste)` permet de mélanger les éléments d'une liste de manière aléatoire.

```
>>> import random  
>>> ma_liste = [1, 2, 3, 4, 5]  
>>> print(random.choice(ma_liste))  
3  
>>> print(ma_liste)  
[1, 2, 3, 4, 5]  
>>> random.shuffle(ma_liste)  
>>> print(ma_liste)  
[3, 4, 1, 2, 5]
```

Le slicing

Le slicing permet de créer une nouvelle liste à partir d'une liste existante en sélectionnant une portion de cette liste.

</> Code Python

```
1 ma_liste = [1, 2, 3, 4, 5]
2 print(ma_liste[1:4])
3 # Affiche: [2, 3, 4]
4 print(ma_liste[:3])
5 # Affiche: [1, 2, 3]
6 print(ma_liste[2:])
7 # Affiche: [3, 4, 5]
```



Les effets de bord sur les listes

- Les listes sont des objets mutables, c'est à dire que leur contenu peut être modifié après leur création.
- Lorsque l'on affecte une liste à une variable, on crée une référence à cette liste. Si on modifie la liste via cette variable, la modification sera visible via toutes les variables qui font référence à cette liste.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste2 = ma_liste
>>> ma_liste2[0] = 10
>>> print(ma_liste)
[10, 2, 3, 4, 5]
>>> print(ma_liste2)
[10, 2, 3, 4, 5]
```

Si on veut faire une vraie copie d'une liste, on peut utiliser la syntaxe suivante :

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste2 = ma_liste[:]
>>> ma_liste2[0] = 10
>>> print(ma_liste)
[1, 2, 3, 4, 5]
>>> print(ma_liste2)
[10, 2, 3, 4, 5]
```

Lorsqu'on utilise des listes dans des fonctions, il faut faire attention aux effets de bord. Si on modifie une liste passée en argument d'une fonction, la modification sera visible à l'extérieur de la fonction.

Dans cet exemple, il n'est pas utile de renvoyer la liste modifiée, car la modification est déjà visible à l'extérieur de la fonction.

</> Code Python

```
1 def ajoute_element(liste, element):
2     liste.append(element)
3
4 ma_liste = [1, 2, 3, 4, 5]
5 ajoute_element(ma_liste, 6)
6 print(ma_liste)           # Affiche: [1, 2, 3, 4, 5, 6]
```



Exercice 6 : Effets de bord et fonctions

Ecrire une fonction `decalage_gauche(liste)` qui décale les éléments de la liste `liste` vers la gauche, en mettant le premier élément à la fin de la liste. La liste doit être modifiée en place.

Code Python

```
def decalage_gauche(liste):  
    ''' Décale les éléments de liste vers la gauche '''  
  
  
  
assert decalage_gauche([1, 2, 3, 4, 5]) == [2, 3, 4, 5, 1]  
assert decalage_gauche([5, 4, 3, 2, 1]) == [4, 3, 2, 1, 5]
```

Exercice 7 : Les listes

Ecrire une fonction qui renvoie une nouvelle liste contenant la somme des éléments de deux listes de même longueur. La fonction doit vérifier que les deux listes ont la même longueur. Si les deux listes ne sont pas de même longueur, la liste la plus courte est complétée par des zéros jusqu'à ce qu'elle ait la même longueur que l'autre liste.

Code Python

```
def somme_listes(L1, L2):  
    ''' Renvoie une nouvelle liste contenant la somme des éléments de L1 et L2 '''  
  
  
  
assert somme_listes([1, 2, 3], [4, 5, 6]) == [5, 7, 9]  
assert somme_listes([1, 2], [3, 4, 5]) == [4, 6, 5]
```


✎ Exercice 8 : Les listes

Écrire une fonction qui modifie une liste en remplaçant chaque élément par la somme de tous les éléments qui sont à sa droite. Par exemple, si la liste est [1, 2, 3, 4], la liste modifiée doit être [10, 9, 7, 4]. La fonction doit modifier la liste en place.

</> Code Python

```
def somme_droite(liste):
    ''' Modifie la liste en remplaçant chaque élément par la somme de tous
        les éléments à sa droite '''

    assert somme_droite([1, 2, 3, 4]) == [10, 9, 7, 4]
    assert somme_droite([5, 4, 3, 2, 1]) == [15, 10, 6, 3, 1]
```



II. Les tuples

Définition 1 : Les tuples

Un **tuple** est une structure de donnée qui contient des éléments ou items identifiés par des indices. Les éléments d'un tuple sont :

- **ordonnés** : chaque élément a une position dans le tuple, appelée indice.
- **séparés par des virgules** : les éléments d'un tuple sont séparés par des virgules.
- **entre parenthèses** : les éléments d'un tuple sont entourés de parenthèses () en python.

Indice i	0	1	2	3	4	5	6	7
Élément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Créer un tuple

</> Code Python

```
1 mon_tuple = (1, 2, 3, 4, 5)
```

</> Code Python

```
1 mon_tuple = ('a', 'b', 'c')
```

```
>>> mon_tuple = (1, 2, 3, 4, 5)
>>> print(type(mon_tuple))      # Un tuple est de type 'tuple'
<class 'tuple'>
```

Propriété 1 : Immutabilité des tuples

Les éléments d'un tuple ne peuvent pas être modifiés après la création du tuple. Un tuple est une structure de donnée immuable.

AP06

ALGORITHMES DE RECHERCHE

I. Algorithmes de recherche naïfs

🔪 Exercice 1 : Recherche d'un élément dans une liste

Ecrire une fonction `recherche_naive(elt, liste)` qui renvoie `True` si l'élément `elt` est présent dans la liste `liste`, et `False` sinon.

🔗 Code Python

```
1 def recherche_naive(elt, liste):
2     ''' Renvoie True si elt est dans liste et False sinon'''
3
4
5
6
7
8
9
10
11
12 assert recherche_naive(3, [1, 2, 3, 4, 5]) == True
13 assert recherche_naive(6, [1, 2, 3, 4, 5]) == False
```

Définition 1 : Notion de complexité d'un algorithme

La **complexité** d'un algorithme est une mesure de la quantité de ressources (temps, mémoire, etc.) que l'algorithme utilise en fonction de la taille de l'entrée. La complexité permet de comparer les algorithmes entre eux et de choisir le plus efficace pour résoudre un problème donné.

Nous allons nous intéresser à la complexité temporelle, (temps d'exécution de l'algorithme en fonction de la taille de l'entrée) exprimée en utilisant la notation **O** (grand O).

Pour étudier la complexité d'un algorithme on se place toujours dans le **pire des cas**, c'est à dire que l'on suppose que l'algorithme doit parcourir tous les éléments de l'entrée pour trouver la solution.

Complexité de la recherche naïve

- Dans le pire des cas, l'algorithme de recherche naïve doit parcourir tous les éléments de la liste pour vérifier si l'élément recherché est présent ou non. (Elément absent ou présent à la fin de la liste)
- Si la liste possède n éléments, alors l'algorithme de recherche naïve doit effectuer n comparaisons. On dira que la complexité est de l'ordre de $O(n)$.
- Si la liste contient 10 éléments, l'algorithme doit effectuer 10 comparaisons dans le pire des cas.
- De manière générale, si la taille de la liste double, le temps d'exécution de l'algorithme de recherche naïve double également. La complexité de l'algorithme de recherche naïve est linéaire.

Définition 2 : Notion de terminaison d'un algorithme

Un algorithme est dit **terminant** s'il s'arrête après un nombre fini d'étapes pour toute entrée donnée.

Un algorithme se termine de façon certaine si :

- Il ne contient pas de boucle infinie.
- S'il fait appel à une boucle infinie (WHILE), il doit y avoir une condition d'arrêt qui est vérifiée à un moment donné.

La condition d'arrêt est appelée **variant de boucle**.

Définition 3 : Le variant de boucle

Un **variant de boucle** est une quantité qui est modifiée à chaque itération d'une boucle et qui permet de garantir la terminaison de l'algorithme.

Pour qu'un variant de boucle soit efficace, il doit être :

- **positif ou nul** pour que la boucle se poursuive ;
- **décroissant** à chaque itération pour que la boucle se rapproche de sa condition d'arrêt.

Exercice 2 : Variant de boucle**</> Code Python**

```

1 def recherche_naive(elt, liste):
2     ''' Renvoie True si elt est dans liste et False sinon'''
3     i = 0
4     while i < len(liste):
5         if liste[i] == elt:
6             return True
7         i += 1
8     return False
9
10 assert recherche_naive(3, [1, 2, 3, 4, 5]) == True
11 assert recherche_naive(6, [1, 2, 3, 4, 5]) == False

```

1. Identifier le variant de boucle de l'algorithme de recherche naïve.
2. Montrer que ce variant de boucle est positif ou nul au début de la boucle, et qu'il est décroissant à chaque itération de la boucle.

Définition 4 : Notion de correction d'un algorithme

Un algorithme est dit **correct** s'il renvoie la bonne solution pour toute entrée donnée.

Pour prouver la correction d'un algorithme, on peut utiliser l'invariant de boucle. Un **invariant de boucle** est une propriété qui est vérifiée avant et après chaque itération d'une boucle. Si l'invariant de boucle est vérifié, alors l'algorithme est correct.

Exercice 3 : Correction d'un algorithme**</> Code Python**

```

1 def fonction_produit(a, b):
2     ''' Renvoie le produit de a par b '''
3     m = 0
4     produit = 0
5     while m < b:
6         produit += a
7         m += 1
8     return produit
9 assert fonction_produit(3, 4) == 12

```

1. Identifier le variant de boucle de la fonction fonction_produit.
2. Montrer que ce variant de boucle est positif ou nul au début de la boucle, et qu'il est décroissant à chaque itération de la boucle. Conclure.
3. Identifier un invariant de boucle de l'algorithme de recherche naïve.
4. Montrer que cet invariant de boucle est vérifié avant et après chaque itération de la boucle. Conclure.

✎ Exercice 4 : division euclidienne

Le principe de l'algorithme de division euclidienne est de soustraire le diviseur du dividende jusqu'à ce que le reste soit inférieur au diviseur. Le nombre de soustractions effectuées correspond au quotient de la division euclidienne, et le reste correspond au reste de la division euclidienne.

Si a est le dividende et b est le diviseur, alors l'algorithme de division euclidienne peut être écrit de la manière suivante :

1. Initialiser le quotient q à 0 et le reste r à a .
2. Tant que le reste r est supérieur ou égal au diviseur b , faire :
 - Soustraire le diviseur b du reste r .
 - Incrémenter le quotient q de 1.
3. Retourner le quotient q et le reste r .

</> Code Python

```
def division_euclidienne(a, b):
    ''' Renvoie quotient et reste de la division euclidienne de a par b '''

    # Python logo watermark

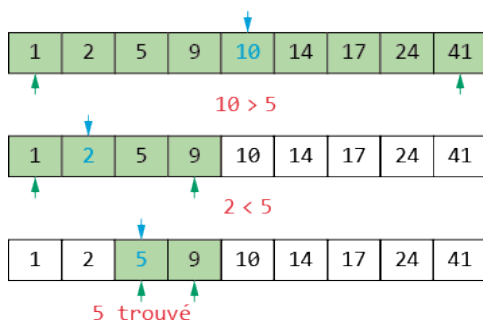
    assert division_euclidienne(10, 3) == (3, 1)
    assert division_euclidienne(10, 4) == (2, 2)
```

1. Identifier le variant de boucle de l'algorithme de division euclidienne.
2. Montrer que ce variant de boucle est positif ou nul au début de la boucle, et qu'il est décroissant à chaque itération de la boucle. Conclure.
3. Identifier un invariant de boucle de l'algorithme de division euclidienne.
4. Montrer que cet invariant de boucle est vérifié avant et après chaque itération de la boucle.
5. Conclure que l'algorithme de division euclidienne est correct.

II. Algorithme de recherche dans une liste triée

Définition 1 : Diviser pour régner

En informatique, la technique de **diviser pour régner** (*divide and conquer*) est une stratégie algorithmique qui consiste à diviser un problème en sous-problèmes plus petits, à résoudre ces sous-problèmes de manière récursive, puis à combiner les solutions pour obtenir la solution du problème initial.



- on compare la valeur du milieu avec la valeur cherchée. $10 > 5$ donc :
- on conserve la moitié gauche du tableau : [1, 2, 5, 9]
- on compare la valeur du milieu avec la valeur cherchée. $2 < 5$ donc :
- on conserve la moitié droite du tableau : [5, 9]
- on compare la valeur du milieu avec la valeur cherchée. La valeur est trouvée.

Exercice 5 : Algorithme de recherche dans une liste triée

Code Python

```

1 def recherche_dichotomique(val, L):
2     i_deb = 0
3     i_fin = len(L) - 1
4     while .....:
5         i_cen = .....
6         if .....:
7             .....
8         elif .....:
9             .....
10        else:
11            .....
12    return False

```

[illegible]

1. Compléter l'algorithme de recherche dichotomique.
2. Compléter la trace de l'algorithme de recherche dichotomique pour la valeur 8 et la liste $[1, 3, 4, 6, 7, 8, 10, 13, 14]$.
3. Identifier le variant de boucle de l'algorithme de recherche dichotomique.
4. Montrer que ce variant de boucle est positif ou nul au début de la boucle, et qu'il est décroissant à chaque itération de la boucle. Conclure.
5. Identifier un invariant de boucle de l'algorithme de recherche dichotomique.
6. Montrer que cet invariant de boucle est vérifié avant et après chaque itération de la boucle. Conclure que l'algorithme de recherche dichotomique est correct.
7. Comparer les complexités des deux algorithmes de recherche.

Définition 2 : Le logarithme

Le **logarithme** est un opérateur mathématique qui a entre autre comme propriété :

$$\log_2(2^k) = k \times \log_2(2) = k \quad \text{et} \quad \log_2(2) = 1$$

Dans notre exemple, lorsque la taille de la liste double, alors le nombre de boucles augmente seulement de 1.

Supposons que la liste comporte n éléments et que n est tel qu'il existe un $k \in \mathbb{N}$ tel que $n = 2^k$.

Etant donné que le nombre de boucles augmente de 1 quand le nombre d'éléments est multiplié par deux alors k est le nombre de boucles nécessaires pour obtenir une liste d'un seul élément.

n	valeur de k	nombre de boucles
1	0	0
2	1	1
4	2	2
8	3	3
16	4	4
...	...	...
n	k	k
2n	k+1	k+1

Pour extraire k en fonction de n , on utilise une fonction nommée logarithme et notée $\log$:

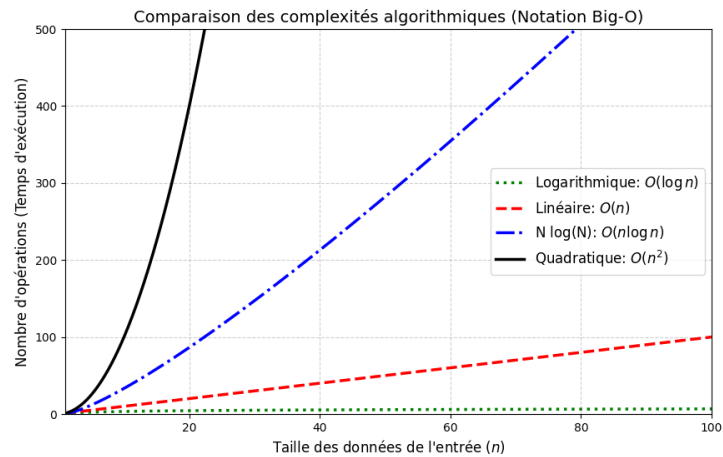
$$n = 2^k \iff \log_2(n) = \log_2(2^k) = k \log_2(2) = k$$

Ainsi $k = \log_2(n)$

k étant le nombre de boucles nécessaires pour obtenir une liste d'un seul élément, alors la complexité est de l'ordre de $O(\log(n))$.

Définition 3 : Complexité logarithmique

Dans l'exemple précédent, on dira que cet algorithme possède une complexité **logarithmique** notée $O(\log(n))$

Définition 4 : Comparaison des complexités**Propriété 1 : Détermination de complexité**

Pour déterminer la complexité d'un algorithme, on peut utiliser les propriétés suivantes :

- Si un algorithme n'effectue pas de boucle, alors sa complexité est de l'ordre de $O(1)$.
- Si un algorithme effectue une boucle FOR, alors sa complexité est de l'ordre de $O(n)$.
- Si un algorithme effectue k boucles FOR imbriquées, alors sa complexité est de l'ordre de $O(n^k)$.
- Si un algorithme effectue une boucle WHILE et à chaque itération la taille du problème est divisée par 2, alors sa complexité est de l'ordre de $O(\log(n))$.

AP07

LES DICTIONNAIRES

I. Introduction

Définition 1 : Les dictionnaires

Un **dictionnaire** est une structure de données qui permet de stocker des paires **clé-valeur**. Les clés sont uniques dans un dictionnaire, tandis que les valeurs peuvent être dupliquées.

En Python, les dictionnaires :

- sont des collections **non ordonnées** d'éléments;
- sont **mutables**, les éléments d'un dictionnaire peuvent être modifiés après sa création;
- sont **indexés par des clés**, qui peuvent être de n'importe quel type de données immuable;
- sont délimités par des **accolades** { } et les paires clé-valeur sont séparées par des **deux-points** :.

</> Code Python

```
1 mon_dictionnaire = {'a': 1, 'b': 2, 'c': 3}
```

Dans cet exemple :

- les **clés** sont 'a', 'b' et 'c';
- les **valeurs** sont 1, 2 et 3;
- la **clé** 'a' est associée à la **valeur** 1;
- le couple **clé-valeur** 'b': 2 signifie que la **clé** 'b' est associée à la **valeur** 2;

Propriété 1 : Les dictionnaires

</> Code Python

```
1 mon_dictionnaire = {'a': 1, 'b': 2, 'c': 3}
2 print(mon_dictionnaire['a']) # Affiche 1
3 print(mon_dictionnaire['b']) # Affiche 2
4 print(mon_dictionnaire['c']) # Affiche 3
5 print(len(mon_dictionnaire)) # Affiche 3
```

II. Itérer sur un dictionnaire

Itérer sur les clés

</> Code Python

```
for cle in dico.keys():
    print(dico[cle])
```

Itérer sur les valeurs

</> Code Python

```
for val in dico.values():
    print(val)
```

Itérer sur les items

</> Code Python

```
for cle, val in dico.items():
    print(cle, val)
```

III. Création de dictionnaires

3 façons de créer un dictionnaire

</> Code Python

```
1 mon_dico = {} # Dictionnaire vide
2 mon_dico = {'a': 1, 'b': 2, 'c': 3} # Création avec des paires clé-valeur
3 mon_dico['a'] = 4 # Création par ajout de clé : valeur
```

Création d'un dictionnaire par itération

</> Code Python

```
1 mon_dico = {}
2 for i in range(5):
3     mon_dico[i] = i**2
4 print(mon_dico) # Affiche {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Création d'un dictionnaire par compréhension

</> Code Python

```
1 mon_dico = {i: i**2 for i in range(5)}
2 print(mon_dico) # Affiche {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Exercice 1 : Création d'un dictionnaire

Ecrire une fonction `longueurs_elts(L)` qui prend en argument une liste de chaînes de caractères `L` et qui renvoie un dictionnaire dont les clés sont les éléments de `L` et les valeurs sont les longueurs des éléments de la liste `L`.

</> Code Python

```
def longueurs_elts(L):
    # A compléter
```

```
assert longueurs_elts(['a', 'ab', 'abc']) == {'a': 1, 'ab': 2, 'abc': 3}
```

IV. Appartenance d'un élément à un dictionnaire

</> Code Python

```
1 mon_dico = {'a': 1, 'b': 2, 'c': 3}
2 print('a' in mon_dico.keys()) # Affiche True
3 print('d' in mon_dico.keys()) # Affiche False
4 print(1 in mon_dico.values()) # Affiche True
5 print(4 in mon_dico.values()) # Affiche False
6 print(('a', 1) in mon_dico.items()) # Affiche True
7 print(('d', 4) in mon_dico.items()) # Affiche False
```

V. Suppression dans un dictionnaire

</> Code Python

```
1 mon_dico = {'a': 1, 'b': 2, 'c': 3}
2 del mon_dico['a'] # Supprime la paire clé-valeur 'a': 1
3 print(mon_dico) # Affiche {'b': 2, 'c': 3}
4 mon_dico.clear() # Supprime toutes les paires clé-valeur du dictionnaire
5 print(mon_dico) # Affiche {}
```


Exercice 2

1. Ecrire une fonction `occurence(sequence)` qui renvoie un dictionnaire dont les clés sont les caractères présents dans `sequence` et leurs valeurs le nombre de fois où elles apparaissent.

</> Code Python

```
def occurence(sequence)->dict:
```



```
assert occurence([4, 2, 3, 4, 1, 4, 3, 4]) == {4: 4, 2: 1, 3: 2, 1: 1}
```

```
assert occurence("abracadabra") == {'a': 5, 'b': 2, 'r': 3, 'c': 1, 'd': 1}
```

2. Ecrire une fonction `plus_frequent(sequence)` qui renvoie le caractère le plus souvent présent dans la `sequence`.

</> Code Python

```
def plus_frequent(sequence):
```



```
assert plus_frequent([4, 2, 3, 4, 1, 4, 3, 4]) == 4
```

```
assert plus_frequent("abracadabra") == 'a'
```

Exercice 3

On considère un dictionnaire donnant la moyenne de quelques étudiants. Partitionner ce dictionnaire en deux sous dictionnaires : un dictionnaire des élèves ayant la moyenne et un dictionnaire des élèves n'ayant pas la moyenne.

</> Code Python

```
def coupe_dico(dico):
```



```
dico={"etudiant1": 15,"etudiant2": 9,"etudiant3": 12,"etudiant4": 5}
```

AP08**TRIS PAR INSERTION ET SÉLECTION****I. Tri par insertion**

<https://www.youtube.com/watch?v=R0a1U37913U>

https://physalgo.fr/Dariusz_Trish/index.html

Ce type de tri est le système que vous utilisez naturellement lorsqu'on vous distribue des cartes une par une et que vous voulez les trier par ordre croissant.

Au fur et à mesure que vous recevez une carte (la **clé**), vous lui trouvez la bonne place en la comparant aux cartes déjà triées. Cette carte va se décaler vers la gauche jusqu'à trouver sa bonne place.

Propriété 1 : Le tri par insertion

1. La liste contenant un seul élément d'indice 0 est triée.
2. La clé est le premier élément non trié;
3. On compare la clé avec l'élément à sa gauche si on n'a pas atteint le bord gauche;
4. Échange des éléments si la clé est inférieure à l'élément de gauche;
5. Retour à l'étape 3
6. La partie triée gagne un élément;
7. Retour à l'étape 2 tant qu'il reste des éléments à trier.

3	8	2	6	5	0	4
3	8	2	6	5	0	4
2	3	8	6	5	0	4
2	3	6	8	5	0	4
2	3	5	6	8	0	4
0	2	3	5	6	8	4
0	2	3	4	5	6	8

Etape 1 :

- au départ la liste triée est constituée de l'élément 3;
- la clé est le premier élément non trié (8).
- la clé est supérieure à l'élément de gauche(3);
- la clé est déposée à l'indice 1 de la liste;
- la partie triée gagne un élément (8);

Etape 2 :

- la liste triée est donc maintenant [3, 8];
- la clé est le premier élément non trié (2).
- la clé est inférieure à l'élément de gauche(8) ⇒ la clé est échangée avec l'élément de gauche (8);
- la clé est inférieure à l'élément de gauche(3) ⇒ la clé est échangée avec l'élément de gauche (3);
- la clé est maintenant tout à gauche de la liste ⇒ la clé est déposée à l'indice 0 de la liste;
- la partie triée gagne un élément (2);

Etape 3 :

- la liste triée est donc maintenant [2, 3, 8];
- la clé est le premier élément non trié (6).
- la clé est inférieure à l'élément de gauche(8) ⇒ la clé est échangée avec l'élément de gauche (8);
- la clé est supérieure à l'élément de gauche(3) ⇒ la clé est déposée à l'indice 2 de la liste;
- la partie triée gagne un élément (6);

Exercice 1

Tri par insertion

1. Compléter le programme suivant afin d'effectuer le tri par insertion. Cette fonction modifie la liste en place et ne renvoie donc rien.

```

</> Code Python
1 def tri_insertion(L):
2     ''' Algorithme de tri par insertion '''
3     n = len(L)
4     for i in range(1, n):
5         # Indice de la clé
6         k = i
7         # Tant que le bord de la liste n'est pas atteint
8         # Et que la clé est inférieure à l'élément de gauche:
9         while k > 0 and L[k-1] > L[k]:
10            # Échange de la clé avec l'élément de gauche
11            L[k], L[k-1] = L[k-1], L[k]
12            # Décalage de la clé vers la gauche
13            k -= 1
14
15 L = [8, 4, 1, 0, -5, 6, 7]           # Liste non triée
16 tri_insertion(L)                   # Tri de la liste
17 assert L == [-5, 0, 1, 4, 6, 7, 8] # La liste est triée

```

2. Effectuer la trace de cet algorithme pour $L=[3, 6, 0, 2]$. Recopier le tableau ci-dessous et le compléter au fur et à mesure de l'exécution de l'algorithme.

Ligne	L[0]	L[1]	L[2]	L[3]	i	k	L[k-1]	L[k]	while

3. Quel est le variant de boucle? L'algorithme se termine-t-il?
4. Quel est l'invariant de boucle? L'algorithme est-il correct?
5. Déterminer la complexité de cet algorithme

II. Tri par sélection

<https://www.youtube.com/watch?v=Ns4TPTC8whw>

https://physalgo.fr/Dariusz_Trish/index.html

Ce type de tri est le système que vous utilisez lorsqu'on vous distribue toutes les cartes d'un coup.

Au départ la liste triée comporte 0 éléments. On cherche le plus petit élément de la liste non triée et on l'échange avec le premier élément de la liste non triée. La partie triée gagne un élément. On recommence tant qu'il reste des éléments à trier.

Propriété 1 : Le tri par sélection

1. la liste triée comporte 0 éléments et la liste non triée n
2. on cherche le minimum de la liste non triée que l'on échange avec le premier élément de la liste non triée;
3. la liste triée augmente de 1 élément
4. on recommence l'étape 1 jusqu'à ce que la liste triée comporte $n - 1$ éléments (le dernier élément est forcément trié).

3	8	2	6	5	0	4
0	8	2	6	5	3	4
0	2	8	6	5	3	4
0	2	3	6	5	8	4
0	2	3	4	5	8	6
0	2	3	4	5	8	6
0	2	3	4	5	6	8

Etape 1 :

- au départ la liste triée est constituée d'aucun élément;
- la liste non triée est [3, 8, 2, 6, 5, 0, 4]
- le premier élément de la liste non triée est 3;
- on cherche le minimum de la liste non triée qui est 0;
- on échange 3 et 0;
- la liste triée gagne un élément (0);
- la liste triée est maintenant [0] et la liste non triée est [8, 2, 6, 5, 3, 4];

Etape 2 :

- la liste triée est donc maintenant [0, 2];
- le premier élément de la liste non triée est 8;
- on cherche le minimum de la liste non triée qui est 3;
- on échange 8 et 3;
- la liste triée gagne un élément (3);
- la liste triée est maintenant [0, 2, 3] et la liste non triée est [8, 6, 5, 4];

Etape 3 :

- la liste triée est donc maintenant [0, 2, 3];
- le premier élément de la liste non triée est 8;
- on cherche le minimum de la liste non triée qui est 4;
- on échange 8 et 4;
- la liste triée gagne un élément (4);
- la liste triée est maintenant [0, 2, 3, 4] et la liste non triée est [8, 6, 5];

Exercice 2

Tri par sélection

1. Compléter le programme suivant afin d'effectuer le tri par sélection. Cette fonction modifie la liste en place et ne renvoie donc rien.

```

</> Code Python
1 def tri_selection(L):
2     ''' Algorithme de tri par sélection '''
3     n = len(L)
4     for i in range(...., ....):
5         # Recherche du plus petit element de la liste non triee
6         mini = i           # indice de plus petit element non triee
7         for j in range(...., ....):
8             if .....:
9                 mini = j     # j est le nouvel indice du minimum
10        # Echange entre L[i] et L[mini]
11        .....
12
13 L = [8, 4, 1, 0, -5, 6, 7]           # Liste non triée
14 tri_selection(L)                   # Tri de la liste
15 assert L == [-5, 0, 1, 4, 6, 7, 8] # La liste est triée

```

2. Effectuer la trace de cet algorithme pour $L=[3, 6, 0, 2]$. Recopier le tableau ci-dessous et le compléter au fur et à mesure de l'exécution de l'algorithme.

Ligne	L[0]	L[1]	L[2]	L[3]	i	mini	j	L[j]	L[mini]	IF

3. L'algorithme se termine-t-il?
4. Quel est l'invariant de boucle? L'algorithme est-il correct?
.....
.....
.....
5. Déterminer la complexité de cet algorithme

AP09

ALGORITHME GLOUTON

Propriété 1 : Algorithme glouton

Les **algorithmes gloutons** répondent à des problèmes d'optimisation. L'idée est qu'à chaque étape de l'algorithme, celui-ci fasse le choix qui lui semble le plus optimal, sans réfléchir à la suite, en espérant que la solution finale au problème soit elle-même la plus optimale. Attention cela n'est pas forcément toujours le cas et dépend du problème à traiter.

I. Problème de rendu de monnaie

L'exemple classique d'application d'un algorithme glouton est le problème du **rendu de monnaie**. L'objectif est de rendre une somme en un nombre de pièces minimal.

Exemple : Problème du rendu de monnaie

On considère une système `SYSTEME = [500, 200, 100, 50, 20, 10, 5, 2, 1]`.

1. Tant que la somme à rendre est supérieure à 0
2. Si la somme à rendre est plus grande que la pièce observée alors :
 - on soustrait cette pièce à la somme à rendre
 - insère la valeur de la somme à rendre dans la liste de choix
3. Sinon on prend la pièce de valeur immédiatement inférieure et on reprend à la ligne 1

Exercice 1

1. Implémenter cette fonction en python

```
</> Code Python
def rendre_monnaie1(systeme:list, somme:int)->list:
    ''' Renvoie la liste des pièces à rendre '''
    i = 0 # Indice de la pièce actuelle
    choix = [] # Liste de choix
    while .....:
        if somme >= .....:
            somme -= .....
            ...
        else:
            ...
    return choix

SYSTEME = [500, 200, 100, 50, 20, 10, 5, 2, 1]
assert rendre_monnaie1(SYSTEME, 674) == [500, 100, 50, 20, 2, 2]
```

2. Compléter la trace de l'algorithme pour somme = 674

somme	i	choix	while	SYSTEME[i]	IF	somme	choix	i
674	0	[]	True	500	True	174	[500]	0
174	0	[500]						

3. Modifier cet algorithme en prenant un système sous forme d'une liste de liste. Chaque sous liste comporte deux éléments : La valeur de la pièce et le nombre de pièces disponibles.

Ex: `SYSTEME2 = [[500,1], [200,1], [100,1], [50,2], [20,2], [10,2], [5,3], [2,6], [1,4]]`
 Attention, il faudra vérifier que le système permet de rendre la somme à rendre.

Remarque

Le système de monnaie a été choisi pour qu'un algorithme glouton soit optimal. Comment payer 9 € avec un système de pièces ou de billets de 1 €, 2 €, 5 € et 10 €. Le choix optimal (moins de pièces et de billet) s'obtient en appliquant l'algorithme glouton (choisir toujours la plus grande des valeurs pour compléter) soit 5 € + 2 € + 2 € (3 pièces). Imaginons qu'il existe un billet de 7 €. L'algorithme glouton n'est plus optimal. En effet si l'on souhaite payer 14 €, il donnerait : 10 € + 2 € + 2 € alors que le meilleur choix serait 7 € + 7 €.

II. Problème du sac à dos

Le problème du sac à dos (aussi noté KP pour *knapsack problem*) est lui aussi un problème d'optimisation.

On dispose de plusieurs objets (chaque objet possède une valeur et un poids). Seulement ce sac ne peut supporter plus d'un certain poids. On va donc devoir choisir des objets et maximiser la valeur totale dans le sac sans toutefois dépasser le poids maximum.

On suppose pour la suite de l'exercice que le sac ne peut supporter que 15 kg maximum

1. Méthode naïve ou exhaustive

Une première méthode consiste à lister toutes les combinaisons possibles d'objets puis de regarder lesquelles ne dépassent pas 15 kg et enfin de regarder laquelle permet d'avoir une somme maximale.

S'il y a n objets, alors il y a 2^n combinaisons possibles. La complexité est donc en $O(2^n)$, c'est un coût exponentiel.

2. Méthode glouton

Un algorithme glouton fait le meilleur choix possible à chaque étape en espérant que la solution finale soit elle-même la meilleure.

Voici cinq objets, leur poids et leur valeur. Vous disposez d'un sac de capacité maximale de 32 kg.

Exercice 2

1. Choisissons un critère de choix d'un objet à mettre dans le sac. Quelle est la valeur du sac si on applique ce critère à chaque choix d'un objet?
2. Définissez un autre critère de choix. Appliquez-le à chaque choix d'objet et donnez la valeur obtenue
3. Même question avec un troisième critère de choix.
4. Y a-t-il une meilleure valeur totale?

Objet	Poids (kg)	Valeur (euros)
A	16	512
B	15	495
C	19	950
D	14	280
E	14	420

Définition 1 : Algorithme glouton

Type d'algorithme dans lequel, à chaque étape, un **choix optimal local** est fait, sans tenir compte de la suite et sans possibilité de retour en arrière (pensez à un glouton qui avale un objet, impossible de revenir en arrière...). On peut donc comparer :

- la méthode **exhaustive** (étude de tous les cas ou force brute) : permet de trouver la meilleure solution à un problème mais en un temps qui peut être considérable;
- une méthode **gloutonne** : ne donne pas forcément la solution optimale à un problème mais donne souvent une solution acceptable en un temps limité. En ce sens, une méthode gloutonne est heuristique, c'est-à-dire une méthode qui permet de trouver une solution convenable mais non forcément optimale à un problème mais en un temps raisonnable en évitant une analyse complète de ce problème.

AP10**ALGORITHME K PLUS PROCHES VOISINS (KNN)****Définition 1 : Algorithme des k plus proches voisins**

Abrégé en **KNN** (K-nearest neighbors), cet algorithme cherche à prédire la classe (ou propriété) d'un élément en fonction de la classe majoritaire de ses k plus proches voisins.

Par exemple, imaginons qu'un pixel soit manquant dans une image. L'algorithme va prédire la couleur de ce pixel en fonction de la couleur des pixels voisins.

L'algorithme KNN est un algorithme d'**apprentissage** (Machine Learning) supervisé. C'est-à-dire qu'à partir d'un ensemble des données étiquetées, il va pouvoir prédire l'étiquette d'une donnée inconnue.

Par exemple, à partir d'un ensemble d'images de chiens et de chats, ce type d'algorithme permet de déterminer si une photo inconnue est celle d'un chien ou d'un chat.

Un autre exemple sont les systèmes de recommandation. Un site marchand peut chercher vos k plus proches voisins en termes d'âge, de lieu, etc., pour vous proposer des articles susceptibles de vous plaire.

I. Principe

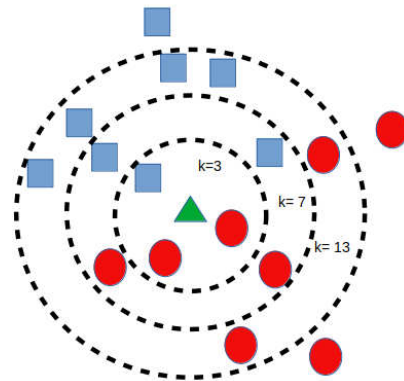
Tout d'abord, il nous faut avoir un ensemble de données d'apprentissage. Chaque donnée :

- appartient à une **classe**.
- est caractérisée par un certain nombre d'**attributs**.

Nous avons un individu non identifié, c'est à dire de **classe inconnue**, dont on possède cependant les caractéristiques. L'objectif est de classer cet individu en regardant ses k plus proches voisins.

II. Exemple

Voici la représentation de deux classes : les carrés et les disques. En représentant leurs caractéristiques graphiquement, on obtient la répartition ci-dessous. Un individu de classe inconnue possédant ces mêmes caractéristiques, est représenté par le triangle.



- En prenant $k=3$, les 3 plus proches voisins sont 2 disques et un carré : on décidera que l'inconnu est un disque.
- En prenant $k=7$, les 7 plus proches voisins sont 4 disques et 3 carrés : on décidera encore que l'inconnu est un disque.
- En prenant $k=13$, les 13 plus proches voisins sont 6 disques et 7 carrés : on décidera alors que l'inconnu est un carré.

Remarque

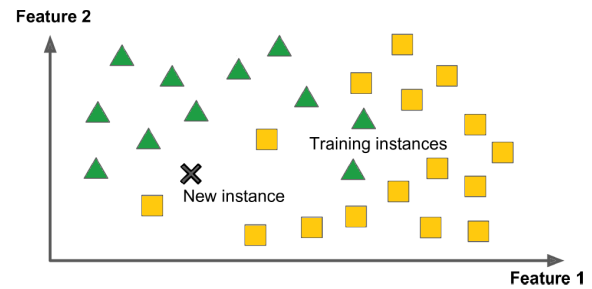
L'algorithme k-NN est un algorithme de décision qui pourrait être résumé par la phrase « dis moi qui sont des voisins et je te dirai qui tu es ».

- En prenant une décision, on fait un choix éclairé mais on ne peut pas être à 100% certain d'avoir fait le bon :
- La définition de la **distance** entre deux classes est particulièrement importante.
- Le choix du k, c'est à dire du nombre de voisins à étudier, est également primordial.

III. Activité 1

On considère un ensemble d'objets dans le plan de deux types différents, des triangles verts et des carrés jaunes.

On ajoute un nouvel objet (la croix que l'on appellera *N*) et on souhaite déterminer à quel type d'objet il appartient.



Plus proche voisin $k = 3$

1. sur la figure précédente, sélectionnez les 3 plus proches voisins du point N.
2. quel type choisir alors pour le point N?

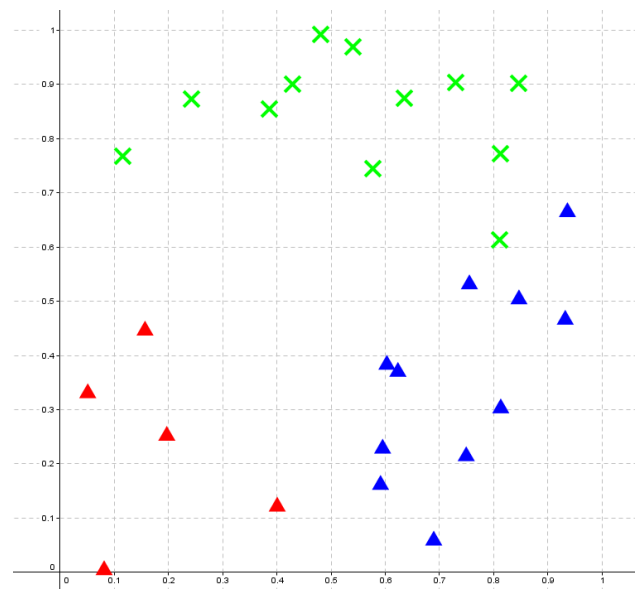
Plus proche voisin $k = 5$

1. sur la figure précédente, sélectionnez les 5 plus proches voisins du point N.
2. quel type choisir alors pour le point N?

IV. Activité 2

On considère la figure ci-dessous avec un nouvel ensemble de points dans le plan de trois classes différentes, et *N* un nouveau point que l'on souhaite colorier.

1. Choisir un point aléatoirement et le placer dans le graphique
2. Déterminer, à l'aide de l'algorithme des k plus proches voisins, la classification de votre nouveau point *N* pour $k=1$, $k=3$ et $k=5$.



Choix du k

Le choix du k est important. En effet, si k est trop petit, on risque de faire du bruit dans les données (on peut être influencé par un voisin qui n'est pas représentatif). Si k est trop grand, on risque d'inclure des voisins qui ne sont pas pertinents pour la classification de l'individu.